

EBIOS RM, expliqué simplement

Pourquoi le logiciel est construit comme ça, ce que veulent dire les mots techniques qu'on entend souvent (Core Engine, mapping, EF...), et comment lire nos schémas, du plus simple au plus complexe.

Aucun terme technique n'est utilisé sans être d'abord expliqué avec une comparaison du quotidien. Si une phrase semble absconse, l'analogie juste avant ou après la démystifie.

01 L'idée de base : trois métiers, trois pièces

Imagine une agence bancaire. Il y a la **salle d'accueil** où tu remplis un formulaire au guichet, le **bureau du back-office** où un employé vérifie que tout est correct et applique les règles de la banque, et le **coffre-fort** où sont gardées les informations, en sécurité, pour toujours. Trois pièces, trois métiers, personne ne fait le travail d'un autre.

EBIOS RM est construit exactement pareil :

- **La salle d'accueil** = ce que tu vois dans ton navigateur (le site web). Elle affiche les informations et récupère ce que tu tapes, mais elle ne décide rien toute seule.
- **Le back-office** = le programme qui tourne sur le serveur. C'est lui qui applique toutes les règles : « on ne peut pas valider un atelier qui n'est pas complet », « une gravité est un nombre entre 1 et 4 », etc.
- **Le coffre-fort** = la base de données. Elle ne fait qu'une chose, mais elle la fait très bien : garder l'information en sécurité et ne jamais la perdre.

POURQUOI SÉPARER LES TROIS ?

Si le guichetier de l'accueil décidait lui-même des règles de la banque, chaque agence appliquerait des règles différentes. En gardant les règles au même endroit (le back-office), toutes les manières d'utiliser l'application — le site web aujourd'hui, une appli mobile demain — respectent exactement les mêmes règles, écrites une seule fois.

02 Un seul immeuble, ou plein de maisons séparées ?

C'est le choix le plus important qu'on a fait, alors prenons le temps de bien l'expliquer.

LE RESTAURANT CONTRE LES FOOD TRUCKS

Imagine un restaurant avec **une seule cuisine** : tous les cuisiniers travaillent dans la même pièce, se parlent directement, partagent les mêmes ingrédients au même endroit. C'est rapide à organiser, facile à surveiller, et un seul chef sait tout ce qui s'y passe.

Maintenant imagine **dix food trucks séparés**, chacun avec sa propre cuisine, garés à des endroits différents, qui doivent se téléphoner pour se coordonner quand un plat a besoin d'un ingrédient préparé par un autre truck. C'est ce qu'on appelle des « microservices ». Ça devient utile quand tu as une *armée* de cuisiniers et que plusieurs équipes différentes doivent travailler chacune dans son coin sans se marcher dessus.

EBIOS RM, aujourd'hui, c'est **une seule cuisine bien organisée en plusieurs postes de travail** (un poste « étude », un poste « rapports »...) — on appelle ça un **monolithe modulaire**. Tout tourne dans un seul programme, mais à l'intérieur, chaque partie a sa zone bien à elle et ne piétine pas les autres.

POURQUOI PAS LES FOOD TRUCKS TOUT DE SUITE ?

Parce qu'on n'a qu'un seul restaurant (une seule équipe, une seule base de clients) pour l'instant. Découper en food trucks maintenant ajouterait beaucoup de téléphones (d'appels réseau) à gérer, pour un problème — trop de monde dans une seule cuisine — qu'on n'a pas encore. Le jour où ce problème apparaît vraiment, on pourra sortir un poste de travail de la cuisine et en faire un food truck séparé, sans tout reconstruire.

03 Le classeur : pourquoi PostgreSQL ?

L'ARMOIRE À TIROIRS AVEC DES RÈGLES STRICTES

Une base de données comme PostgreSQL, c'est une armoire immense où chaque dossier a un tiroir précis, étiqueté, avec des **règles qui empêchent de mal ranger** : impossible de mettre « douze » dans le tiroir qui n'accepte que des nombres entre 1 et 4, impossible de ranger un dossier « bien support » sans indiquer à quelle « valeur métier » il appartient. Ces règles s'appellent des *contraintes*, et elles existent pour qu'aucune erreur de rangement ne se glisse jamais, même si le programme qui range a un bug.

PostgreSQL a été choisi parce que le contenu d'une étude EBIOS RM (les risques, les biens à protéger...) est une donnée **précieuse et structurée** — elle a besoin d'un classeur sérieux, pas d'un tas de post-it. C'est aussi un classeur gratuit, très utilisé, et qui sait parfaitement gérer « deux personnes qui essaient de ranger dans le même tiroir en même temps » sans tout casser.

04 Le bureau du back-office : pourquoi C# / .NET ?

LE CLERC POINTILLEUX

C# est un langage qui oblige à déclarer à l'avance « ce champ ne peut être qu'un nombre », « celui-ci ne peut jamais être vide ». C'est comme un employé de back-office très pointilleux qui refuse un formulaire mal rempli *avant même* de commencer à le traiter, plutôt que de découvrir l'erreur en cours de route. Pour une application où une erreur de gravité mal codée pourrait fausser toute une analyse de risque, ce genre de rigueur vaut mieux qu'un langage plus permissif.

.NET 8 est la version actuelle et robuste de cet environnement, largement utilisée par des grandes entreprises et administrations — ce n'est pas un choix expérimental.

05 Ce qu'il y a précisément dans le back-office

La section 1 a dit « le back-office applique les règles ». Mais le back-office lui-même n'est pas un seul gros bloc — c'est plusieurs bureaux spécialisés qui se passent le dossier à tour de rôle, chacun avec un métier précis. Ce sont exactement les boîtes que tu vois dans le diagramme de composants. Voyons-les une par une, dans l'ordre où un dossier les traverse.

Endpoint / API — le guichet d'accueil

Un « endpoint », c'est une adresse précise où envoyer une demande précise — comme un guichet numéroté dans une administration. Il n'y a pas UN guichet pour tout : il y a un guichet « créer une étude », un autre « valider un atelier », un autre « obtenir le rapport ». L'ensemble de ces guichets s'appelle une **API** (le comptoir complet de tous les guichets disponibles). La vitrine (le site web) sait à quel guichet frapper pour chaque demande.

DANS NOS DIAGRAMMES

Le rectangle « Endpoints » dans le composant « Présentation ».

Handler / Application — le chef de dossier

Une fois la demande reçue au guichet, quelqu'un doit orchestrer les étapes : vérifier que le dossier existe, demander au bon bureau spécialisé de faire son travail, puis ranger le résultat. C'est le rôle du « handler » (littéralement : « celui qui gère »). Il ne connaît pas les règles métier lui-même — il sait juste dans quel ordre appeler les bons spécialistes.

DANS NOS DIAGRAMMES

Le rectangle « Handlers de cas d'usage » dans le composant « Application ».

Core Engine (Domain) — le bureau des règles métier

C'est LE bureau spécialiste d'EBIOS RM. Il connaît par cœur toutes les règles de la méthode — « une étude ne peut être validée que si elle est en cours », « la gravité va de 1 à 4 » — et refuse tout ce qui les enfreint. Ce bureau ne sait *rien d'autre* : il ignore totalement à quoi ressemble le site web, et il ignore totalement comment les dossiers sont rangés dans le coffre-fort. C'est fait exprès : le jour où on change de classeur (une autre base de données) ou de vitrine (un autre site web), ce bureau ne bouge pas d'un pouce, parce que les règles EBIOS RM restent les mêmes.

DANS NOS DIAGRAMMES

Le rectangle « Etude – agrégat racine » et « Services de domaine », dans « Core Engine – Domain ». C'est aussi ce que montre le diagramme de classes en entier.

Repository — le coursier des archives

Le bureau des règles métier ne va jamais lui-même fouiller dans le coffre-fort — ce n'est pas son travail, et ça le rendrait dépendant d'un classeur précis. À la place, il demande à un coursier : « va me chercher le dossier de l'étude numéro 12 » ou « range ce dossier modifié ». Le coursier (le **repository**) est le seul à savoir où et comment ranger physiquement les choses.

DANS NOS DIAGRAMMES

Le rectangle « Repositories EF Core » dans « Infrastructure ».

Mapping / Entity Framework (EF) Core — le traducteur automatique

Voici un détail important : le bureau des règles métier parle un langage riche (« une Étude, avec un nom, un statut, et la capacité de se faire Valider »), alors que le coffre-fort ne comprend que des lignes et des colonnes bien rangées dans des tableaux. Quelqu'un doit traduire entre les deux, dans les deux sens, à chaque fois. Cette traduction automatique s'appelle le **mapping** (littéralement : « la mise en correspondance »), et l'outil qui la fait tout seul, sans qu'on ait à l'écrire à la main à chaque fois, s'appelle **Entity Framework Core** (EF Core). Grâce à lui, le coursier n'a même pas besoin d'écrire des ordres de rangement compliqués : il donne le dossier tel quel à EF Core, qui sait tout seul dans quel tiroir et sous quelle forme le ranger.

EXEMPLE CONCRET DE MAPPING

Un objet **Etude** en mémoire (Nom = "Banque X", Statut = EnCours) ↔ une ligne dans la table **etudes** de PostgreSQL (colonne Nom = 'Banque X', colonne Statut = 'EnCours'). EF Core fait cet aller-retour automatiquement.

Reporting — l'imprimeur officiel

Un bureau à part, qui ne travaille que sur des dossiers **déjà validés et tamponnés** — jamais sur un brouillon en cours de modification. C'est lui qui produit le rapport PDF final. S'il travaillait sur un brouillon, le rapport pourrait changer sous tes yeux pendant que quelqu'un d'autre modifie encore l'étude — ce serait dangereux pour un document censé faire foi.

DANS NOS DIAGRAMMES

Le rectangle « Génération de rapports » dans « Reporting ».

Remets ces six bureaux bout à bout, dans l'ordre, et tu obtiens exactement le chemin que suit une demande : **guichet** → **chef de dossier** → **bureau des règles** → **coursier** → **traducteur** → **coffre-fort** — puis le même chemin en sens inverse pour répondre. C'est précisément ce que raconte le diagramme de composants, et dans le détail, le diagramme de séquence.

06 La salle d'accueil : pourquoi React ?

LA VITRINE INTERACTIVE

React permet de construire une page web qui se met à jour toute seule, comme une vitrine de magasin où les prix changent automatiquement dès qu'on modifie le stock en arrière-boutique — sans devoir recharger toute la page. TypeScript ajoute la même rigueur que C# côté vitrine (« ce champ est forcément un texte, pas un nombre »), Vite est l'outil qui prépare rapidement la vitrine avant de l'ouvrir au public, et Tailwind est la boîte de peinture et de règles qui rend tout visuellement cohérent sans réinventer le style à chaque nouvelle page.

07 La règle pour l'assistant IA de demain

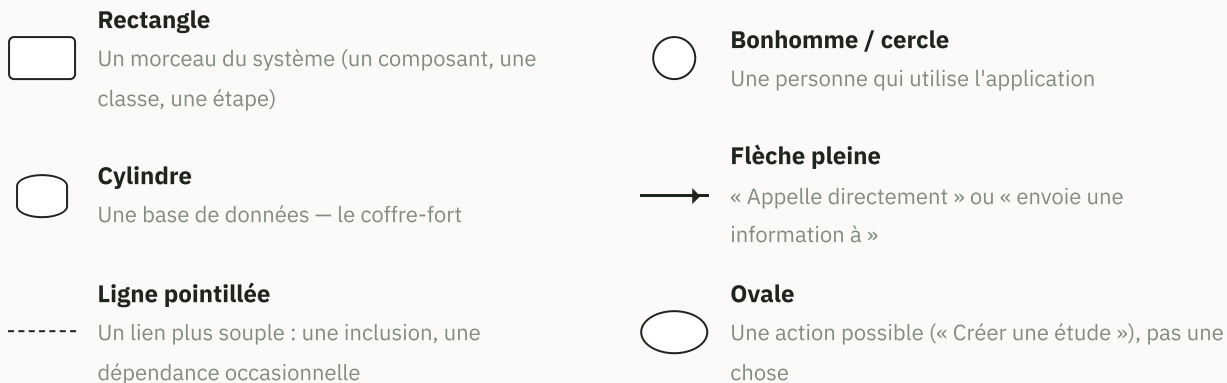
LE STAGIAIRE QUI PROPOSE, MAIS NE SIGNE JAMAIS

Un stagiaire dans une banque peut suggérer une idée — « je pense que ce risque devrait être noté plus grave » — mais il n'a jamais le droit de modifier un dossier officiel tout seul. Seul un employé habilité peut lire sa suggestion et décider de l'appliquer ou non.

C'est exactement la règle actée pour la future assistance IA d'EBIOS RM : elle pourra **proposer**, jamais **écrire directement** dans une étude. Un humain doit toujours accepter explicitement la suggestion pour qu'elle devienne réelle. Cette règle est déjà dessinée dans les schémas d'architecture — le rectangle « Assistance IA » du diagramme de composants ne parle qu'au « chef de dossier » (l'Application), jamais directement au bureau des règles ni au coffre-fort — même si l'IA elle-même n'est pas encore construite.

08 Comment lire nos schémas, du plus simple au plus complexe

Tous les diagrammes utilisent le même alphabet visuel de base (des boîtes, des flèches, des étiquettes), mais chaque *type* de diagramme raconte une histoire différente. On va les prendre dans l'ordre du plus facile à lire au plus dense, pour construire ta lecture petit à petit.



1 — LE PLUS SIMPLE

Le diagramme d'états : une histoire avec un début et des étapes

C'est le plus facile : lis-le littéralement comme une bande dessinée, de gauche à droite ou de haut en bas. Le point noir plein, c'est le **début de l'histoire**. Chaque rectangle est une **situation possible**. Chaque flèche est un **événement qui fait passer d'une situation à une autre**, et son étiquette dit ce qui doit se passer pour que ça arrive.

Prends notre diagramme « Statut global de l'étude » : le point noir mène à **Brouillon** (l'étude vient d'être créée). La flèche « premier atelier démarré » mène à **EnCours**. La flèche « les 5 ateliers sont Validés » mène à **Finalisée**. Et une flèche revient en arrière si « un atelier est rouvert ». Tu viens de lire tout le cycle de vie d'une étude, sans rien savoir du code.

2

Le diagramme de séquence : qui parle à qui, et dans quel ordre

Chaque acteur ou bureau reçoit sa propre **ligne verticale en pointillé** qu'on appelle une « ligne de vie » — imagine des couloirs de piscine qui descendent la page. Le **temps s'écoule du haut vers le bas**. Une flèche horizontale qui part d'un couloir vers un autre est un **message envoyé à ce moment précis**. Un rectangle gris posé sur plusieurs couloirs à la fois est une **note** qui explique quelque chose d'important à cet instant de l'histoire.

Dans « Validation d'un atelier » : le couloir « Responsable » envoie « Valide l'atelier » au couloir « Frontend », qui relaie une requête au couloir « Endpoint », et ainsi de suite en descendant — jusqu'à la note grise qui rappelle que la validation et le snapshot se font ensemble, dans une seule transaction.

3

Le diagramme de cas d'utilisation : qui a le droit de faire quoi

Les bonhommes à gauche sont les **humains** qui utilisent la plateforme. Le grand rectangle à droite, c'est **la plateforme elle-même** — tout ce qui est dedans est une fonctionnalité qu'elle offre, dessinée en **ovale**. Une ligne simple entre un bonhomme et un ovale veut dire « cette personne peut faire cette action ». Une flèche en pointillés avec l'étiquette « include » veut dire « cette action en déclenche automatiquement une autre, sans qu'on ait besoin de le demander séparément ».

« Demander une suggestion IA » pointe en pointillé vers « Accepter / rejeter une suggestion » — ce qui te dit, sans lire une ligne de code, que demander une suggestion mène toujours, ensuite, à devoir l'accepter ou la rejeter. À l'inverse, remarque qu'il n'y a *pas* de flèche « include » entre « Valider un atelier » et « Générer un rapport » : valider fige automatiquement un instantané (un « snapshot »), mais générer le rapport PDF reste une action séparée, à la demande — les deux ne sont pas la même chose.

4

Le diagramme de composants : les pièces du bâtiment, imbriquées

Ici, une boîte peut en contenir d'autres, comme des pièces dans un bâtiment, dans une rue, dans une ville. Le grand rectangle « EbiosRM.Api » est le bâtiment ; à l'intérieur, chaque pièce (Présentation, Application, Core Engine...) contient elle-même ses propres meubles (Endpoints, Handlers...). Une flèche qui sort d'une pièce vers une autre veut dire « cette pièce a le droit d'appeler celle-là ». C'est exactement la liste des six bureaux qu'on a détaillés à la section 5 — ce schéma en est le plan d'étage.

5

Le diagramme de déploiement : où ça tourne, pour de vrai

Ressemble au diagramme de composants, mais ne parle plus du *logiciel* : il parle des **machines physiques** (ou virtuelles) qui le font tourner. Chaque grande boîte 3D est un ordinateur distinct — ton navigateur sur ton poste, le serveur qui exécute l'API, le serveur qui héberge la base de données. Les flèches entre les boîtes montrent le protocole réseau utilisé (HTTPS, par exemple) plutôt qu'un appel de code direct.

Le diagramme de classes : la carte d'identité complète du métier

C'est le plus riche, donc le plus utile une fois qu'on sait le lire. Chaque rectangle est coupé en **trois étages** : en haut le **nom** de la chose (ex. Etude), au milieu ses **caractéristiques** (ex. Nom : string), en bas ses **actions possibles** (ex. ValiderAtelier(numero)).

Signe moins

- Réserve à l'intérieur — les autres bureaux ne peuvent pas y toucher directement

Signe plus

- + Les autres bureaux ont le droit d'utiliser cette action

Losange plein



- « Fait partie de » — ne peut pas exister sans son propriétaire



Losange vide

- « Regroupe », mais peut exister sans lui

Les petits chiffres près des lignes (comme 1 et 0..*) s'appellent des **cardinalités** : ils répondent à « combien, exactement ? ». 1 veut dire « exactement un ». 0..* veut dire « zéro, un, ou autant qu'on veut ».

Lis la ligne entre **Etude** et **ValeurMetier** : un losange plein côté Etude, « 1 » d'un côté et « 0..* » de l'autre, avec l'étiquette « contient ». Traduction en phrase normale : « **Une étude contient zéro, une, ou plusieurs valeurs métier, et une valeur métier ne peut pas exister sans son étude.** » C'est tout le principe des cardinalités : une ligne du diagramme se relit toujours comme une phrase complète.

09 Nos diagrammes, expliqués un par un

La section précédente t'a appris la *grammaire* de chaque type de schéma. Ici, on ouvre chacun de nos six diagrammes et on le lit en entier, littéralement, boîte par boîte et flèche par flèche — dans le même ordre que les diagrammes officiels (numérotés Diagramme 01 à 06 dans le PDF des diagrammes UML).

DIAGRAMME 05

Les états — le plus simple, en entier

Ce diagramme est en réalité trois petites histoires côte à côte.

« **Cycle d'un atelier (1 à 5)** » : le point noir part vers un rectangle appelé **NonDemarre**, avec l'étiquette « Etude créée » sur la flèche — ça veut dire qu'un atelier commence toujours non démarré, dès que l'étude existe. Une flèche part de **NonDemarre** vers **EnCours**, étiquetée **DemarrerAtelier(numero)** : c'est l'action qui fait passer d'un état à l'autre. De **EnCours**, une flèche mène à **Valide**, étiquetée **ValiderAtelier(numero)**, avec en dessous une précision « (crée une nouvelle version de snapshot) ». Et une dernière flèche revient de **Valide** vers **EnCours**, étiquetée **RouvrirAtelier(numero)** — la boucle est bouclée, un atelier validé peut redevenir « en cours ». Une note grise accrochée à **Valide** rappelle que chaque passage par cet état fige une nouvelle version, sans jamais écraser l'ancienne.

« **Statut global de l'étude** » : même grammaire, quatre états cette fois. Le point noir mène à **Brouillon** (étiquette : **Creer(nom, perimetre, mission)**, l'action de création). De **Brouillon**, une flèche « premier atelier démarré » mène à **EnCours**. De **EnCours**, une flèche « les 5 ateliers sont Validés » mène à **Finalisee**. Et une flèche revient de **Finalisee** vers

EnCours, étiquetée « cycle stratégique (reprise atelier 1) ou cycle opérationnel (reprise atelier 2) » — pas juste « un atelier est rouvert » : la documentation officielle montre que l'atelier 5 boucle vraiment vers l'atelier 1 (tout reprendre) ou vers l'atelier 2 (reprendre seulement la partie opérationnelle). La note accrochée à **Finalisee** précise que ce statut n'est jamais saisi à la main : il est *calculé* à partir de l'état des 5 ateliers, et que ce retour en arrière n'est pas une correction d'erreur — c'est la méthode elle-même qui fonctionne par cycles, pas en ligne droite.

« **Suivi d'une mesure du plan de traitement du risque** » : la plus courte des trois, trois états en ligne. Le point noir mène à **ALancer**, étiqueté `AjouterMesure( . . . )` — dès qu'une mesure de sécurité est décidée dans le plan, elle démarre à ce statut. Une flèche « mise en œuvre démarrée » mène à **EnCours**, puis « mise en œuvre achevée » mène à **Termine**. Ce sont les trois statuts exacts trouvés dans le tableau de suivi de la documentation officielle (le plan s'appelait autrefois « PACS » — Plan d'Amélioration Continue de la Sécurité — jusqu'à EBIOS RM 1.5, qui l'a rebaptisé « Plan de traitement du risque » ; les statuts n'ont pas changé, seul le nom du plan). Et surtout : ce suivi est *individuel*, mesure par mesure, pas un seul statut global pour tout le plan.

DIAGRAMME 04

La séquence de validation, message par message

Neuf couloirs verticaux, de gauche à droite : le bonhomme **Responsable / Validateur**, puis **Frontend, Endpoint, ValiderAtelierHandler, ServiceValidationCompleture, Etude (agrégat), ServiceCreationSnapshot, IEtudeRepository**, et le cylindre **PostgreSQL** tout à droite. En descendant la page, voici exactement ce qui se passe, dans l'ordre :

1. Responsable → Frontend : « Valide l'atelier » — le clic sur le bouton.
2. Frontend → Endpoint : « POST .../ateliers/{numero}/valider » — la vraie requête envoyée sur le réseau.
3. Endpoint → Handler : « Handle(Commande) » — le guichet transmet le dossier au chef de dossier.
4. Handler → Repository : « ObtenirParIdAsync(id) » — le chef de dossier demande le dossier complet de l'étude.
5. Repository → PostgreSQL : « SELECT étude + enfants ».
6. *PostgreSQL → Repository* : « lignes » — flèche en pointillé, donc une *réponse*, pas une nouvelle demande.
7. *Repository → Handler* : « Etude (agrégat complet) ».
8. Handler → ServiceValidationCompleture : « VerifierAsync(id) » — avant de toucher à quoi que ce soit, le chef de dossier vérifie que l'atelier a bien tout ce qu'il faut (par exemple : au moins un scénario de risque évalué, pour l'atelier 5).
9. *ServiceValidationCompleture → Handler* : « complet ? » — si la réponse est non, tout s'arrête ici : réponse 400, aucune écriture en base. Les étapes suivantes ne se produisent que si c'est oui.
10. Handler → Etude : « ValiderAtelier(numero) » — le chef de dossier demande au bureau des règles de faire la vérification.
11. *Etude → Handler* : « EnCours → Validé » — la transition a réussi.
12. Handler → ServiceCreationSnapshot : « CreerSnapshot(...) ».
13. *ServiceCreationSnapshot → Handler* : « Snapshot (version n+1) ».

14. Une **note grise** posée entre le couloir Handler et le couloir PostgreSQL : « Transaction unique — étude et snapshot réussissent ou échouent ensemble ». C'est le moment le plus important du schéma.
15. Handler → Repository : « MettreAJourAsync(...) ».
16. Repository → PostgreSQL : « BEGIN · UPDATE · INSERT · COMMIT » — les quatre mots d'une vraie transaction de base de données, écrits littéralement sur la flèche.
17. PostgreSQL → Repository : « OK ».
18. Repository → Handler : « succès ».
19. Handler → Endpoint : « résultat ».
20. Endpoint → Frontend : « 200 OK » — le code qui veut dire « tout s'est bien passé ».
21. Frontend → Responsable : « Atelier validé » — ce que la personne voit finalement s'afficher à l'écran.

Vingt-et-une étapes, mais deux choses à retenir : les étapes 8 et 9 peuvent tout interrompre avant la moindre écriture (un atelier incomplet n'est jamais validé), et les étapes 15 et 16 (la vraie écriture dans le coffre-fort) ne peuvent jamais se produire qu'ensemble, grâce à la note de l'étape 14.

DIAGRAMME 01

Les cas d'utilisation, bonhomme par bonhomme

Trois bonhommes empilés à gauche, un grand rectangle « Plateforme EBIOS RM » à droite, avec onze ovales dedans.

L'Administrateur (en haut) n'a que deux lignes qui partent de lui : une vers l'ovale « Gérer le référentiel », une vers « Gérer comptes et rôles ». C'est tout ce qu'il peut faire — il ne touche jamais aux études elles-mêmes.

L'Analyste risque (au milieu) a cinq lignes : « Démarrer un atelier », « Demander une suggestion IA », « Créer une étude », « Saisir les données d'un atelier », et « Consulter l'historique des versions ».

Le Responsable / Validateur (en bas) a cinq lignes aussi, dont une partagée avec l'Analyste (« Consulter l'historique des versions ») : « Valider un atelier », « Rouvrir un atelier validé », et deux longues courbes qui redescendent vers « Accepter / rejeter une suggestion » et « Générer un rapport ».

Une seule flèche en pointillés, étiquetée « include », complète le tableau : de « Demander une suggestion IA » vers « Accepter / rejeter une suggestion » (demander une suggestion mène toujours, ensuite, à devoir l'accepter ou la rejeter). Il n'y en a volontairement *pas* entre « Valider un atelier » et « Générer un rapport » : valider fige un instantané automatiquement, mais générer le rapport PDF reste une action séparée, à la demande — ce n'est pas la même chose, donc pas un « include ».

Une précision honnête : ces 3 bonhommes ne sont *pas* les vraies personnes qui participent à un atelier EBIOS RM en vrai. La documentation officielle réunit Direction, Métiers, RSSI, DSI, et parfois un spécialiste cybersécurité selon l'atelier — des rôles de réunion, pas des comptes

dans un logiciel. Nos 3 acteurs sont une simplification volontaire, pour que la plateforme reste utilisable sans reproduire toute la hiérarchie d'une entreprise.

DIAGRAMME 03 Les composants, pièce par pièce

Tout en haut, le rectangle « Frontend SPA — React + TypeScript + Vite ». Une flèche « HTTPS / JSON » descend vers un grand rectangle « EbiosRM.Api », qui contient six pièces :

- **Présentation**, avec deux meubles à l'intérieur : « Endpoints » et « Authentification ». Une flèche va d'Endpoints vers Authentification.
- **Application**, avec « Handlers de cas d'usage ». Endpoints pointe vers ce meuble.
- **Core Engine — Domain**, avec « Etude — agrégat racine » et « Services de domaine ». Les Handlers pointent vers Etude, qui pointe elle-même vers les Services de domaine.
- **Infrastructure**, avec « Repositories EF Core ». Les Handlers pointent aussi vers ce meuble.
- **Reporting**, avec « Génération de rapports ». C'est Endpoints qui pointe directement vers lui — pas les Handlers, une route à part.
- **Assistance IA**, avec « Suggestions ». Deux flèches en pointillés la relient aux Handlers : une étiquetée « commande explicite » qui part des Handlers vers Suggestions, une étiquetée « suggestion » qui revient de Suggestions vers les Handlers.

En bas, hors du grand rectangle, le cylindre « PostgreSQL ». Deux flèches y arrivent : une depuis « Repositories EF Core », une depuis « Génération de rapports » — les deux seuls chemins autorisés vers le coffre-fort.

DIAGRAMME 06 Le déploiement, machine par machine

Trois grandes boîtes empilées verticalement, chacune représentant un ordinateur distinct, pas un simple bout de logiciel.

« **Poste utilisateur** » en haut, contenant « Navigateur — exécute le bundle React » : c'est ton ordinateur ou ton téléphone. Une flèche « HTTPS » (le protocole sécurisé du web) descend vers « **Serveur applicatif** », qui contient « EbiosRM.Api (Kestrel) — .NET 8 » — Kestrel est simplement le nom du petit moteur qui fait tourner le programme .NET en continu sur ce serveur. Une seconde flèche, « Npgsql, TLS » (Npgsql est le nom du traducteur réseau que .NET utilise pour parler à PostgreSQL ; TLS veut dire que la conversation est chiffrée), descend vers « **Serveur de données** », qui contient le cylindre « PostgreSQL 16 ».

Trois machines, deux flèches, aucune boîte supplémentaire : c'est exactement la simplicité voulue pour une V1, expliquée à la section 2.

DIAGRAMME 02 Les classes, packaging par packaging — le plus long, gardé pour la fin

Cinq paquetages, un par atelier. Leur position sur la page ne suit pas forcément l'ordre 1→5 (le moteur de mise en page les replace pour limiter les croisements de lignes) — la « vue d'ensemble » séparée, avec seulement 5 boîtes dans le bon ordre et ses deux flèches de bouclage, précède ce diagramme détaillé pour cette raison. Ce qui suit a été entièrement revérifié contre la documentation méthodologique officielle — deux classes qu'on avait ratées au premier passage (**CheminAttaque** et **ScenarioDeRisque**) ont été ajoutées, et plusieurs cardinalités corrigées.

Atelier 1 – Cadrage

La classe **Etude** est en haut, avec ses caractéristiques (Nom, Perimetre, Mission, Statut) et ses actions (Creer, DemarrerAtelier, ValiderAtelier, RouvrirAtelier, AjouterValeurMetier...). Un losange plein la relie à **ValeurMetier** (étiquette « contient », 1 vers 0..*). **ValeurMetier** se relie à **EvenementRedoute** (« redoute », toujours en losange plein). La relation vers **BienSupport** (« repose sur »), elle, n'a pas de losange — une simple ligne, avec « 1..* » côté **ValeurMetier** et « 0..* » côté **BienSupport** : un même bien support (un serveur, par exemple) peut très bien être partagé par plusieurs valeurs métier, ce n'est donc pas une possession exclusive. **Etude** se relie aussi à **SocleSecurite** (« applique », 1 vers 0..1 — une étude a au plus un seul socle), qui se relie à **ReferentielApplicable** (« évalue »).

Atelier 2 – Sources de risque

Etude se relie directement à **CoupleSourceRisqueObjectifVise** (« identifie ») et à **PartiePrenante** (« implique »). Une flèche en pointillé (une dépendance, pas une possession) relie **CoupleSourceRisqueObjectifVise** au rectangle « «domain service» ServiceCalculPertinence » : c'est ce service qui calcule automatiquement le champ **Pertinence**, jamais saisi à la main. De même, **PartiePrenante** dépend de « «domain service» ServiceCalculNiveauDangerosite » pour calculer sa **NiveauDangerositeCalculee** (« Dangerosité », terme officiel depuis EBIOS RM 1.5, remplace « Menace ») — dont on connaît maintenant la vraie formule officielle : $(Dépendance \times Pénétration) / (Maturité \times Confiance)$, revérifiée par calcul sur l'exemple du cours. L'analyste peut retenir une valeur différente du calcul (jugement d'expert, justification obligatoire), sans jamais écraser la valeur calculée.

Atelier 3 – Scénarios stratégiques

Etude se relie à **ScenarioStrategique** (« projette »). Deux flèches pointent vers **ScenarioStrategique** : **CoupleSourceRisqueObjectifVise** « donne naissance à » un scénario — et ici la cardinalité est stricte, « 1 » des deux côtés, pas « 0..* » : la documentation officielle dit littéralement « 1 couple SR/OV = 1 scénario stratégique ». **EvenementRedoute**, lui, « est ciblé par » 0 à plusieurs scénarios (un même événement redouté peut être visé par plusieurs attaques différentes).
Nouveauté : **ScenarioStrategique** ne traverse plus directement les parties prenantes — il se décline d'abord en un ou plusieurs **CheminAttaque** (losange plein, « se décline en », 1..* — un scénario a toujours au moins un chemin d'attaque identifié). C'est le *chemin d'attaque*, pas le scénario dans son ensemble, qui traverse l'écosystème : losange creux vers **PartiePrenante**, « 0..* » des deux côtés — un chemin peut traverser zéro partie prenante (une attaque directe, sans tiers), une seule, ou plusieurs, exactement comme avant, mais porté par la bonne classe cette fois. Et chaque partie prenante traversée génère un **EvenementIntermediaire** nommé (un losange plein depuis **CheminAttaque**, « génère », 0..* — zéro si le chemin est direct) : c'est la trace de ce qui se passe concrètement à cette étape, par exemple « création d'un canal d'exfiltration depuis l'infrastructure du prestataire ». Un losange plein relie **PartiePrenante** elle-même (pas **ScenarioStrategique**) à

MesureEcosysteme (« réduit sa dangerosité via ») : ces mesures ciblent une partie prenante précise de l'écosystème, pas un scénario dans son ensemble.

Atelier 4 – Scénarios opérationnels

ScenarioOperationnel se relie à **ModeOperatoire** (« compose », losange plein, 1..* — au moins une étape). Nouveauté : chaque mode opératoire se décompose à son tour en **ActionElementaire** (« décompose en », 1..*), le geste technique le plus fin, chacune portant sa propre **Phase** (CONNAÎTRE/RENTRE/TROUVER/EXPLOITER) — et chaque action élémentaire cible précisément un **BienSupport** de l'atelier 1 (flèche simple « cible ») : la boucle est bouclée, l'attaque finit toujours par toucher un bien concret identifié dès le premier atelier. Une dépendance en pointillé relie ScenarioOperationnel à « «domain service» ServiceCalculVraisemblance », qui calcule automatiquement la VraisemblanceCalculee de chaque mode opératoire — sur l'échelle officielle à 4 niveaux (peu vraisemblable à certain ou déjà produit). Là aussi, l'analyste peut retenir une vraisemblance différente avec justification, sans perdre la valeur calculée.

Atelier 5 – Traitement du risque

C'est ici que la correction la plus importante a eu lieu. On avait mis **RisqueResiduel**, produit directement par ScenarioOperationnel — comme si le risque résiduel était encore une classe de plus dans la chaîne. En réalité, la documentation officielle dit : « 1 scénario de risque = 1 chemin d'attaque et son scénario opérationnel associé ». Autrement dit, **ScenarioDeRisque** n'ajoute rien de nouveau après coup : c'est le *même* chemin d'attaque et le *même* scénario opérationnel, simplement regardés ensemble. D'où les deux losanges pleins qui partent de ScenarioDeRisque : un vers CheminAttaque (« repose sur », toujours 1), un vers ScenarioOperationnel (« repose sur », 0..1 — il peut ne pas encore exister tant que l'atelier 4 n'a pas fait son travail). Attention, une nuance importante entre les deux niveaux de risque : le niveau *initial* n'est *pas* une caractéristique stockée sur ScenarioDeRisque — il se recalcule à la volée à partir de la Gravité (portée par l'événement redouté, jamais dupliquée) et de la Vraisemblance du scénario opérationnel, via la dépendance vers « «domain service» ServiceCalculNiveauRisque ». Le niveau *résiduel*, lui, exige une vraie nouvelle saisie de l'analyste — une Gravité et une Vraisemblance résiduelles, ré-estimées après application des mesures — d'où les attributs réellement stockés sur cette classe : c'est la seule vraie information nouvelle que porte ScenarioDeRisque.

ScenarioDeRisque porte aussi l'acceptation formelle du risque résiduel par la Direction — un nom de propriétaire du risque, un nom de validateur sécurité, et, uniquement si le risque résiduel reste au niveau Élevé, un sponsor exécutif nommé et une justification écrite (exigence ISO/CEI 27005:2022, et pratique observée dans la documentation officielle : une direction peut tout à fait décider de maintenir un risque élevé plutôt que de le traiter davantage, à condition de le documenter).

Enfin, Etude se relie à **PlanTraitementRisque** (« décide », 0..1 — un seul plan par étude). Il se relie en losange plein à **MesureTraitementRisque** (« planifie »). C'est cette fois la *mesure*, pas le plan dans son ensemble, qui se relie à **ScenarioDeRisque** (« traite ») — et par une flèche simple avec des cardinalités « plusieurs vers plusieurs » (0..* d'un côté, 1..* de l'autre) : une même mesure (par exemple « authentification multi-facteurs sur tous les accès distants ») peut couvrir plusieurs scénarios de risque à la fois. Chaque mesure porte un Axe (les 4 axes officiels : Gouvernance, Protection, Défense, Résilience), un Responsable — qui l'exécute, un rôle différent du propriétaire d'un actif (atelier 1) ou du propriétaire d'un risque (ci-dessus) — et un Statut suivi individuellement (voir le 3^e diagramme d'états, section précédente). Ce plan s'appelait « PACS » (Plan d'Amélioration Continue de la Sécurité) jusqu'à EBIOS RM 1.0 ; EBIOS RM 1.5 l'a rebaptisé « Plan de traitement du risque », le nom officiel désormais utilisé partout dans ce projet.

Transverse – snapshot et rapport

Quatre classes qui ne sont dans aucun des cinq paquetages, parce qu'elles s'appliquent identiquement à chacun d'eux (ou, pour la dernière, à l'étude entière) : Etude dépend de « «domain service» ServiceCreationSnapshot », qui crée un **SnapshotAtelier** à chaque validation. Ce snapshot dépend à son tour de « «domain service» ServiceGenerationRapport », mais *séparément, à la demande* — pas automatiquement au moment de la validation. C'est le même mécanisme que celui détaillé étape par étape dans le diagramme de séquence. Une fois les 5 ateliers validés, un quatrième service, « «domain service» RapportSyntheseGlobaleService », lit les 5 SnapshotAtelier d'une même étude (une flèche « 5 » sur la dépendance) pour construire le document de synthèse remis à la Direction — chiffres clés, cartographie des risques avant/après, avancement du plan de traitement, registre d'acceptation. C'est un document *distinct* du rapport d'atelier 5, pas une variante : il raconte les 5 ateliers ensemble, pas un seul.

Lu d'un bout à l'autre, ce diagramme raconte maintenant fidèlement le déroulement complet d'EBIOS RM : une source de risque et un objectif visé donnent naissance à exactement un scénario stratégique, qui se décline en chemins d'attaque traversant (ou non) l'écosystème, chacun donnant lieu à un scénario opérationnel — ensemble, ils forment un scénario de risque, évalué avant puis après le plan de traitement du risque, avec une acceptation formelle du résiduel par la Direction. Le diagramme de classes n'est donc pas juste une liste de boîtes : c'est la méthode EBIOS RM elle-même, dessinée, et désormais vérifiée ligne par ligne contre sa documentation d'origine.

10 La boîte à outils

PIÈCE	OUTIL	POURQUOI CELUI-LÀ, EN CLAIR
Vitrine (frontend)	React	Une page web qui se met à jour toute seule, sans rechargement.
	TypeScript	Le même genre de rigueur que C#, côté vitrine.
	Vite	Prépare et sert la vitrine rapidement pendant qu'on la construit.
	Tailwind CSS	La boîte à outils de style, pour que tout se ressemble visuellement.
Back-office	C# / .NET 8	Le clerc pointilleux qui applique les règles métier sans erreur.
	ASP.NET Core	La partie qui sait recevoir et répondre aux demandes du navigateur — les guichets (endpoints).
	Entity Framework Core	Le traducteur automatique entre le langage du bureau des règles et celui du classeur (voir section 5).
Coffre-fort	PostgreSQL 16	L'armoire à tiroirs stricte qui garde toutes les études EBIOS RM.

PIÈCE	OUTIL	POURQUOI CELUI-LÀ, EN CLAIR
Rapports	QuestPDF	Transforme une étude figée en document PDF présentable.
Environnement	Docker Compose	Démarre le classeur (PostgreSQL) de façon identique sur n'importe quel ordinateur.
Sécurité (à venir)	Identité + OIDC	Vérifie qui se connecte, conçu pour brancher plus tard un système d'authentification d'entreprise.

11 Ce qu'on a décidé ensemble, en clair

Décision 1

Une « étude » protège ses propres dossiers. On ne peut plus ajouter un bien support à une étude en passant par-dessous — il faut toujours demander à l'étude elle-même, qui vérifie que tout est cohérent.

Décision 2

Des petits assistants simples, pas une grosse machine. Chaque action (créer une étude, valider un atelier...) a son propre petit bout de code dédié, sans ajouter un système compliqué qui ferait tout passer par un seul gros tuyau.

Décision 3

Une carte d'identité simple aujourd'hui, échangeable demain. On construit d'abord un système de connexion basique, pensé dès le départ pour être remplacé plus tard par un système d'entreprise (comme ceux des banques ou administrations) sans tout reconstruire.

Décision 4

Le catalogue ISO 27001 attend son tour. Il reste, pour l'instant, dans la vitrine plutôt que dans le coffre-fort — ce n'est pas urgent tant qu'on n'en a pas vraiment besoin pour calculer quelque chose d'important.

12 Et après ?

Rien de tout ça n'est gravé dans le marbre. Le principe qui guide toutes ces décisions est simple : **construire ce dont on a besoin maintenant, de façon assez propre pour pouvoir ajouter la suite sans tout casser.** Le jour où EBIOS RM aura vraiment beaucoup d'utilisateurs en même temps, on pourra ajouter un « portier » devant plusieurs cuisines (un reverse proxy devant plusieurs instances de l'API) sans changer une ligne des règles métier — précisément parce qu'on les a gardées bien séparées dès le début.